

2차전지 AI 실무 · 번외 실습

Gemini 프롬프트로 조립하는 배터리 로트 품질 예측 GUI

3·4장에서 배운 정제 규약과 Random Forest 를 프로그램 하나로
프롬프트 14개 · 파이썬 파일 5개 · 2시간 · 화면 PyQt5 + Plotly
데이터: battery_process_data.csv

선수 학습: 2장 파이썬 기초, 3장 전처리, 4장 회귀·분류 (수강 완료 기준)

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

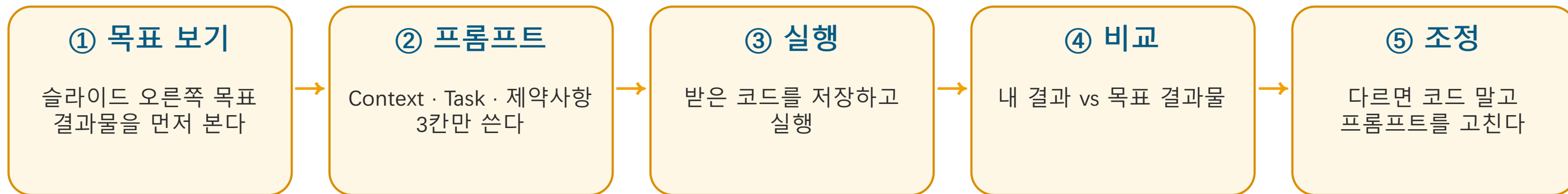
학습 목표

2시간 뒤에는 이 다섯 가지를 할 수 있다

- Gemini 에게 역할·맥락·제약을 주고, 작은 단위로 코드를 받아 내는 프롬프트를 쓴다
- 교재 3.7.1 정제 규약(9999 → NaN, 물리 범위, 중앙값, 중복 제거)을 함수로 구현한 코드를 검증한다
- Random Forest 회귀(용량)·분류(불량)를 학습하고 R^2 ·RMSE·재현율·임계값의 의미를 프로그램에서 확인한다
- tkinter 화면에 파일 열기 → 학습 → 새 로트 입력 → 예측 → 중요도 차트를 붙여 실행 파일로 조립한다
- 생성된 코드를 교재 수치와 대조해 틀린 곳을 찾고, 오류 메시지를 프롬프트로 되돌려 고친다

진행 방식 - 프롬프트를 쪼개서 던지고, 조각을 조립한다

한 번에 "프로그램 만들어 줘" 는 금지 - 함수 한두 개 단위로 요청한다

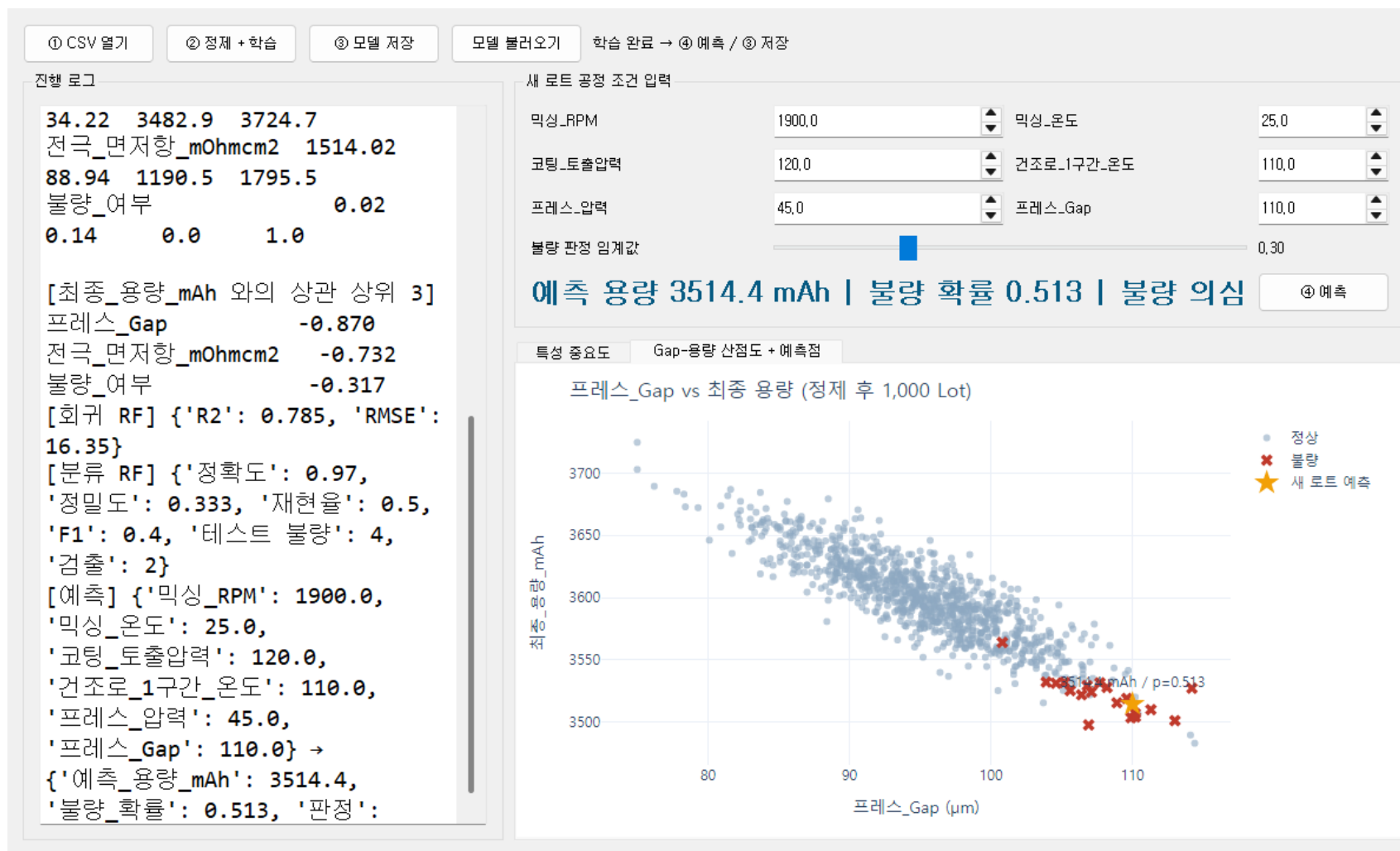


- 프롬프트 14개 = 1부 데이터(4) + 2부 모델(5) + 3부 GUI(3) + 조립(1) + 확장(1)
- 목표와 다르면 코드를 손으로 고치지 말고, 제약사항을 한 줄 더해 다시 요청한다 - 이것이 이 실습의 핵심 연습
- Gemini 대화 창은 하나를 끝까지 쓴다(앞 프롬프트의 맥락을 기억하게)
- 막히면 강사용 정답 코드와 diff 로 비교한다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

완성 프로그램 미리보기 - 배터리 로트 품질 예측기

2시간 뒤 각자 이 화면을 만든다 - 정답 코드의 실제 실행 화면



[그림 X-1] 완성 화면(정답 코드 실행, Gap 110·RPM 1900 예측)

- ① 열기 · ② 정제+학습: 왼쪽 로그에 정제 결과, R².재현율
- 입력 칸 6개 + 임계값 슬라이더 → ④ 예측
- 결과 한 줄: 예측 용량 · 불량 확률 · 판정
- Plotly 탭: 특성 중요도 / Gap-용량 산점도 + 예측점(별)
- ③ 저장 / 불러오기: joblib

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프로그램 구조 - 파일 5개, 프롬프트 14개

3장은 cleaning.py 로, 4장은 model.py 로 들어간다

파일	역할	교재 근거	프롬프트
data_io.py	CSV 읽기, 열 10개 검증, 한 줄 요약	2.10 pandas 기초	2
cleaning.py	정제 4단계(9999→NaN, 물리 범위, 중앙값, 중복) + EDA 요약	3.7.1 정제 규약, 3.11 상관	3·4
model.py	분할(stratify), RF 회귀·분류, 임계값 평가, 중요도, joblib, 새 로트 예측	4.2·4.4·4.12·4.13, 5.9·5.13	5~9
gui_qt.py	PyQt5 창: 버튼 4개·로그·QDoubleSpinBox 폼·QSlider·Plotly 탭 2개(중요도, Gap-용량 산점도+예측점)	(GUI·Plotly 는 이 실습에서 새로 배움)	10~12
main_qt.py	QApplication 생성 + MainWindow 표시	-	13

[표 X-2] 파일별 역할과 교재 근거

- 의존 방향: main → gui → (data_io, cleaning, model). gui 에는 계산 코드를 넣지 않는다(교재 6.13 리포팅 분리 원칙)
- 이 구조를 프롬프트 1 에서 Gemini 에게 먼저 알려 주면, 이후 답변이 파일 단위로 정리되어 나온다

프롬프트는 3칸 - Context · Task · 제약사항

길게 설명하지 말고, 목표 결과물과 다르면 제약사항을 한 줄씩 더한다

칸	쓰는 것	예 (프롬프트 3)
Context	왜 필요한가, 지금 상황 (1~2줄)	원본에 에러 코드 9999 와 물리적으로 불가능한 값이 섞여 있다
Task	무엇을 만들라 (함수·파일 이름)	clean_data(df) 와 cleaning_report(before, after) 를 작성해라
제약사항	지켜야 할 규칙·숫자·형식 (/ 로 구분)	9999 → NaN / 범위 밖 → NaN / 결측 → 중앙값 / 원본 수정 금지

[표 X-3] 프롬프트 3칸

- 역할 부여·출력 형식·긴 설명은 쓰지 않는다 - 프로젝트 맥락은 첫 프롬프트에서 한 번만 알려 준다
- 결과가 목표와 다르면: 다른 점 한 줄 + 제약사항 한 줄 → 다시 요청 (예: 테스트 불량이 0건이다 / 불량_여부로 stratify 할 것)
- 오류가 나면: 오류 메시지 전체를 붙이고 원인 한 줄과 고친 파일 전체를 요청한다

1부

데이터 - 3장 전처리를 코드로

프롬프트 1~4 · 25분 · data_io.py, cleaning.py
체크포인트 ①: 콘솔 수치가 교재 3.7.1 · 3.11 과 일치

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 1. 프로젝트 알려 주기

파일: 없음

[Gemini 에 입력]

Context

battery_process_data.csv(1,000행)로 배터리 로트 품질 예측 데스크톱 앱을 단계별로 만든다. 열: Lot_ID, 믹싱_RPM, 믹싱_온도, 코팅_토출압력, 건조로_1구간_온도, 프레스_압력, 프레스_Gap, 최종_용량_mAh, 전극_면저항_mOhmcm2, 불량_여부

Task

이 정보를 기억하고, 앞으로 요청하는 파일의 코드만 작성해라.

제약사항

- 지금은 "준비됨" 만 답할 것
- 파일: data_io.py, cleaning.py, model.py, gui_qt.py, main_qt.py
- 라이브러리: pandas, scikit-learn, joblib, PyQt5, plotly

목표 결과물 - 이렇게 나오면 성공

준비됨

■ 다른면: 코드나 긴 설명이 나오면 → 제약에 "코드 작성 금지" 를 더한다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 2. CSV 읽기

파일: data_io.py

[Gemini 에 입력]

Context

앱이 쓸 공정 데이터를 CSV 에서 읽어야 한다.

Task

load_csv(path) 와, 읽은 결과를 한 줄로 요약하는 describe_loaded(df) 를 작성해라.

제약사항

- encoding utf-8-sig
- 필수 열 10개가 없으면 ValueError
- 요약 형식은 오른쪽 결과와 같게

목표 결과물 - 이렇게 나오면 성공

행 1000개, 열 10개, 결측 5칸, 불량 19건

- 다르면: 결측,불량 수가 다르면 → 요약 형식을 결과 문장 그대로 붙여 넣어 요구한다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 3. 정제 규약 적용

파일: cleaning.py

[Gemini 에 입력]

Context

원본에 에러 코드 9999 와 물리적으로 불가능한 값이 섞여 있다(교재 3.7.1).

Task

clean_data(df) 와, 정제 전후를 비교하는 cleaning_report(before, after) 를 작성해라.

제약사항

- 9999 → NaN
- 범위 밖 → NaN(믹싱_온도 10~50, 건조로_1구간_온도 80~160, 프레스_압력 20~80)
- 결측 → 중앙값
- Lot_ID 중복 제거
- 원본 수정 금지

목표 결과물 - 이렇게 나오면 성공

[정제 결과] 1000행 → 1000행, 결측 5칸 → 0칸
에러 코드 9999: 5건 → NaN → 중앙값 대체
믹싱_온도 물리 범위(10~50) 위반: 0건
건조로_1구간_온도 물리 범위(80~160) 위반: 5건
프레스_압력 물리 범위(20~80) 위반: 0건

- 다른면: 결측이 0칸이 아니면 → 처리 순서(9999 → 범위 → 중앙값 → 중복)를 제약에 번호로 적는다

프롬프트 4. EDA 요약

파일: cleaning.py

[Gemini 에 입력]

Context

정제한 데이터의 분포와, 용량에 영향을 주는 인자를 한눈에 보고 싶다.

Task

eda_summary(df) 를 cleaning.py 에 추가해라.

제약사항

- 수치 열의 mean·std·min·max
- 최종_용량_mAh 와 상관 절댓값 상위 3개
- 문자열로 반환
- 기존 함수 수정 금지

목표 결과물 - 이렇게 나오면 성공

[기술통계]

	mean	std	min	max
믹싱_RPM	1747.68	79.13	1458.0	2004.0
믹싱_온도	24.88	1.52	20.4	29.4
코팅_토출압력	120.27	8.14	95.5	144.5
건조로_1구간_온도	109.99	3.90	99.0	122.4
프레스_압력	39.91	3.98	28.0	52.0
프레스_Gap	95.30	6.09	75.0	114.4
최종_용량_mAh	3599.00	34.22	3482.9	3724.7
전극_면저항_mOhmcm2	1514.02	88.94	1190.5	1795.5
불량_여부	0.02	0.14	0.0	1.0

[최종_용량_mAh 와의 상관 상위 3]

프레스_Gap	-0.870
전극_면저항_mOhmcm2	-0.732
불량_여부	-0.317

- 다르면: 상관 1위가 프레스_Gap 이 아니면 → 정제 후 df 로 계산하라고 적는다

체크포인트 ① - 콘솔에서 데이터 단계 확인

전원이 같은 숫자를 보고 있어야 2부로 간다

```
> python check_data.py # 배포 폴더의 확인 스크립트

# check_data.py 내용 (7 줄)
import data_io, cleaning
raw = data_io.load_csv('../data/battery_process_data.csv')
print(data_io.describe_loaded(raw))
df = cleaning.clean_data(raw)
print(cleaning.cleaning_report(raw, df))
print(cleaning.eda_summary(df))

# 출력
행 1000개, 열 10개, 결측 5칸, 불량 19건
[정제 결과] 1000행 → 1000행, 결측 5칸 → 0칸
에러 코드 9999: 5건 → NaN → 중앙값 대체
건조로_1구간_온도 물리 범위(80~160) 위반: 5건
[최종_용량_mAh 와의 상관 상위 3]
프레스_Gap -0.870 / 전극_면저항 -0.732 / 불량_여부 -0.317
```

항목	기대값	교재 근거
행 수 / 결측	1000 / 5 → 0	3.2 · 3.7.1
9999 건수	5건	3.3.1
건조로 범위 위반	5건(=9999 건)	3.3.4
믹싱_온도_프레스_압력 위반	0건	3.7.1
상관 1위	프레스_Gap -0.870	3.11

[표 X-5] 체크포인트 ① 기대값

- 다른면: 정제 4단계 순서(9999 → 범위 → 중앙값 → 중복)를 먼저 의심
- eda_summary 의 상관은 정제 후 df 로 계산해야 한다

2부

모델 - 4장 Random Forest 회귀·분류

프롬프트 5~9 · 35분 · model.py
체크포인트 ②: R^2 · 재현율 · 특성 중요도 확인

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 5. 학습/테스트 분할

파일: model.py

[Gemini 에 입력]

Context

모델 실력을 공정하게 재려면 데이터를 나눠야 한다. 불량은 2% 뿐이다.

Task

입력 6개(믹싱_RPM~프레스_Gap)·타깃 2개(최종_용량_mAh, 불량_여부)를 정의하고 split_data(df) 를 작성해라.

제약사항

- 테스트 20%
- random_state 42
- 불량_여부 로 stratify
- 회귀·분류 타깃을 한 번에 분할

목표 결과물 - 이렇게 나오면 성공

학습 (800, 6) / 테스트 (200, 6)
테스트 불량 4건 (학습 불량 15건)

■ 다른면: 테스트 불량이 4건이 아니면 → stratify 를 제약에 명시한다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 6. Random Forest 회귀

파일: model.py

[Gemini 에 입력]

Context

공정 조건으로 최종 용량(mAh)을 예측하고 싶다.

Task

train_regressor 와 evaluate_regressor 를 추가해라.

제약사항

- RandomForestRegressor(n_estimators=300, random_state=42)
- R² 소수 4자리, RMSE 소수 2자리 딕셔너리 반환

목표 결과물 - 이렇게 나오면 성공

{'R2': 0.785, 'RMSE': 16.35}

- 다른면: R² 가 0.75~0.80 밖이면 → n_estimators·random_state 를 확인한다(버전 차이로 소수점은 달라도 된다)

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 7. Random Forest 분류와 임계값

파일: model.py

[Gemini 에 입력]

Context

불량(2%)을 놓치지 않는 것이 정확도보다 중요하다.

Task

train_classifier 와, 임계값을 받는 evaluate_classifier(clf, X, y, threshold) 를 추가해라.

제약사항

- class_weight="balanced"
- predict 대신 predict_proba $\geq$ threshold 로 판정
- 정확도.정밀도.재현율.F1.검출 건수 반환

목표 결과물 - 이렇게 나오면 성공

임계값 0.5 → 정확도 0.975 재현율 0.000 F1 0.000 검출 0/4
임계값 0.3 → 정확도 0.970 재현율 0.500 F1 0.400 검출 2/4
임계값 0.1 → 정확도 0.975 재현율 0.750 F1 0.545 검출 3/4

- 다른면: 임계값을 바꿔도 결과가 같으면 → predict_proba 사용을 제약에 다시 적는다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 8. 특성 중요도

파일: model.py

[Gemini 에 입력]

Context

어떤 공정 인자가 용량을 좌우하는지 알고 싶다.

Task

feature_importance(model) 를 추가해라.

제약사항

- feature_importances_ 를 입력 열 이름이 붙은 Series 로
- 내림차순

목표 결과물 - 이렇게 나오면 성공

프레스_Gap	0.786
코팅_토출압력	0.095
프레스_압력	0.033
믹싱_RPM	0.031
믹싱_온도	0.030
건조로_1구간_온도	0.027

- 다른면: 열 이름 대신 숫자가 나오면 → "입력 열 이름을 인덱스로" 를 제약에 넣는다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 9. 저장·불러오기·새 로트 예측

파일: model.py

[Gemini 에 입력]

Context

학습한 모델을 다시 쓰고, 새 로트 조건을 넣어 바로 판정하고 싶다.

Task

save_models, load_models, predict_one(reg, clf, values, threshold) 를 추가해라.

제약사항

- joblib 파일 하나에 두 모델
- values 는 {열 이름: 값}
- 예측 용량·불량 확률·판정("정상"/"불량 의심") 반환

목표 결과물 - 이렇게 나오면 성공

```
저장: battery_rf_models.joblib
기본 조건 → {예측_용량_mAh: 3593.1, '불량_확률': 0.0, '판정': '정상'}
Gap 110·RPM 1900 → {예측_용량_mAh: 3514.4, '불량_확률': 0.513, '판정': '불량 의심'}
```

- 다른면: 경고(feature names)가 뜨면 → 1행 DataFrame 으로 예측하라고 적는다

체크포인트 ② - 모델 수치 확인

숫자는 버전에 따라 소수점이 다르다 - 범위와 방향을 본다

항목	이 PC 실행값	교재 값(4장)	판단 기준
RF 회귀 R ² / RMSE	0.785 / 16.35 mAh	0.7535 / 15.99 (4.4 기본)	R ² 0.75~0.80, RMSE 15~17
분류 임계값 0.5	재현율 0.000, 검출 0/4	정확도만 높음(4.12)	정확도 ≥ 0.97 인데 검출 0 이면 정상 동작
분류 임계값 0.3	재현율 0.500, 검출 2/4	임계값 하향 시 재현율 ↑ (4.13)	0.5 보다 검출이 늘어야 함
분류 임계값 0.1	재현율 0.750, 검출 3/4	정밀도와 맞교환(4.13)	정밀도가 내려가는 것을 함께 본다
특성 중요도 1:2위	프레스_Gap 0.79, 코팅_토출압력 0.10	5.9 MDI 와 같은 순위	1위가 Gap 이 아니면 FEATS 순서 의심
새 로트 예측(기본값)	3593 mAh, 정상	평균 용량 3599 (부록 A)	Gap 110 이면 약 3510 으로 하락

[표 X-6] 체크포인트 ② 기대값

- 확인 명령: python check_model.py (배포 폴더의 확인 스크립트 - 분할·회귀·분류 3개 임계값·중요도·새 로트 예측을 한 번에 출력)
- 다른면: split_data 의 random_state=42·stratify, RF 의 n_estimators=300·class_weight='balanced', evaluate_classifier 의 predict_proba 를 순서대로 확인

3부

GUI - PyQt5 + Plotly 화면에 붙이기

프롬프트 10~12 · 35분 · gui_qt.py
체크포인트 ③: 화면에서 열기 → 학습 → 예측 동작

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 10. 앱 뼈대: 버튼과 로그

파일: gui_qt.py

[Gemini 에 입력]

Context

지금까지 만든 기능을 버튼으로 실행하는 창이 필요하다.

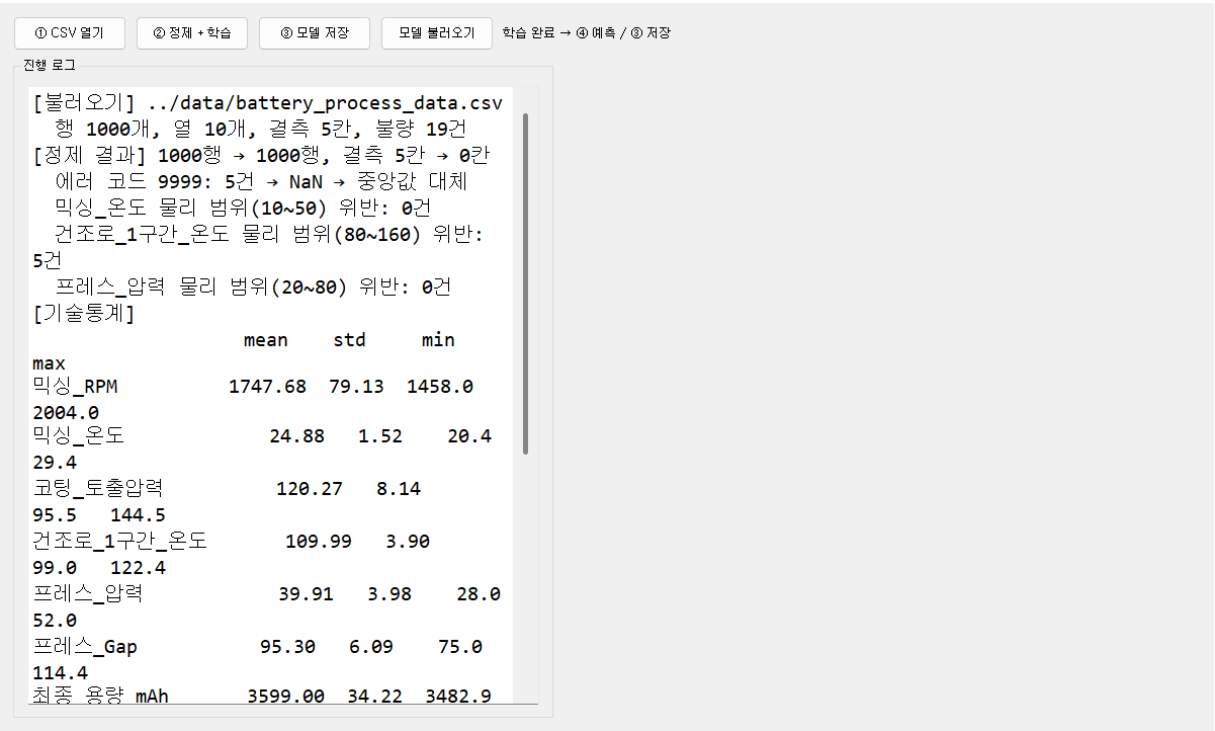
Task

MainWindow 에 버튼 4개(① CSV 열기, ② 정제 + 학습, ③ 모델 저장, 모델 불러오기)와 진행 로그를 만들고 각 버튼을 data_io·cleaning·model 함수에 연결해라.

제약사항

- PyQt5
- 오른쪽 영역은 비워 둔다
- 모든 결과는 로그에 출력
- 순서가 틀리면 경고창

목표 결과물 - 이렇게 나오면 성공



- 다른: 창 모양이 다르면 → 오른쪽 화면을 보여 주듯 배치(위 버튼 줄, 왼쪽 로그)를 글로 적는다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 11. 새 로트 입력과 예측

파일: gui_qt.py

[Gemini 에 입력]

Context

현장에서 새 로트 조건을 입력해 바로 판정해야 한다.

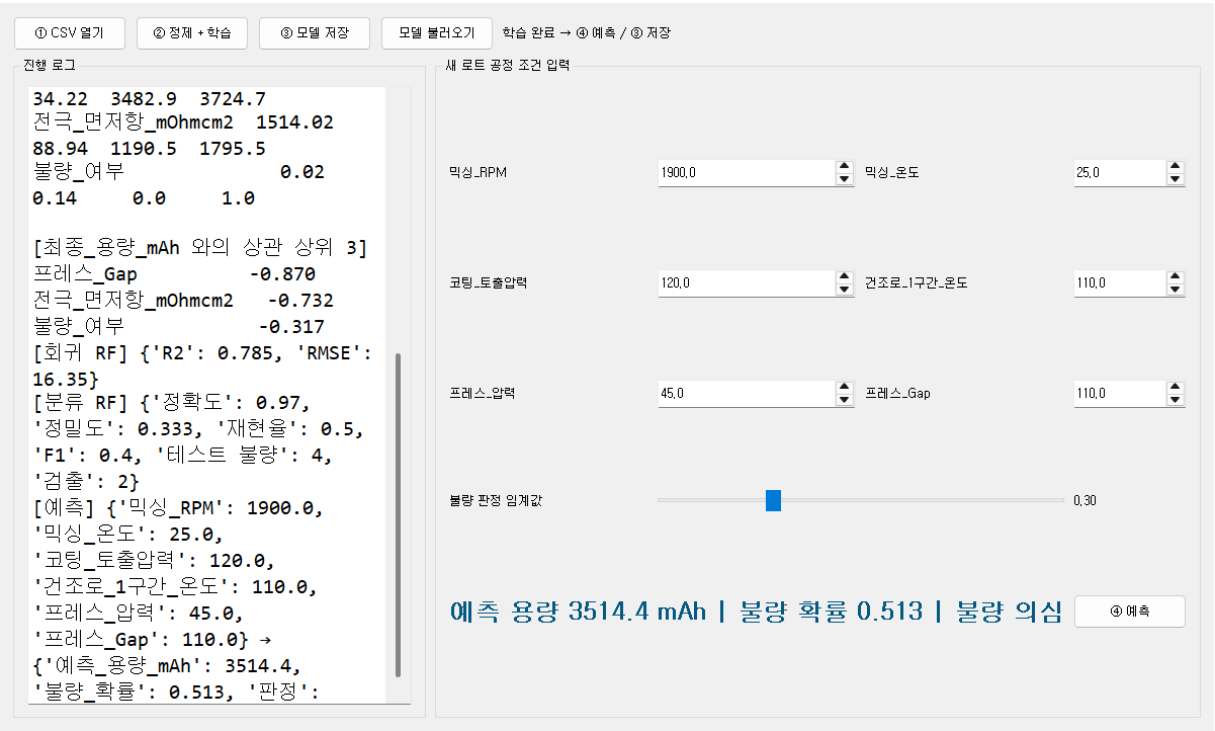
Task

오른쪽에 입력 칸 6개, 불량 판정 임계값 슬라이더, ④ 예측 버튼과 결과 문구를 추가해라.

제약사항

- 숫자 입력은 QDoubleSpinBox(값 범위 지정)
- 임계값 0.05~0.95, 기본 0.30
- 결과는 굵은 파란 글씨 한 줄

목표 결과물 - 이렇게 나오면 성공



- 다른면: 임계값을 바꿔도 판정이 그대로면 → 예측할 때 슬라이더 값을 넘기라고 적는다

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 12. Plotly 차트 탭

파일: gui_qt.py

[Gemini 에 입력]

Context

결과를 그림으로 보여 줘야 현장에 설명하기 쉽다.

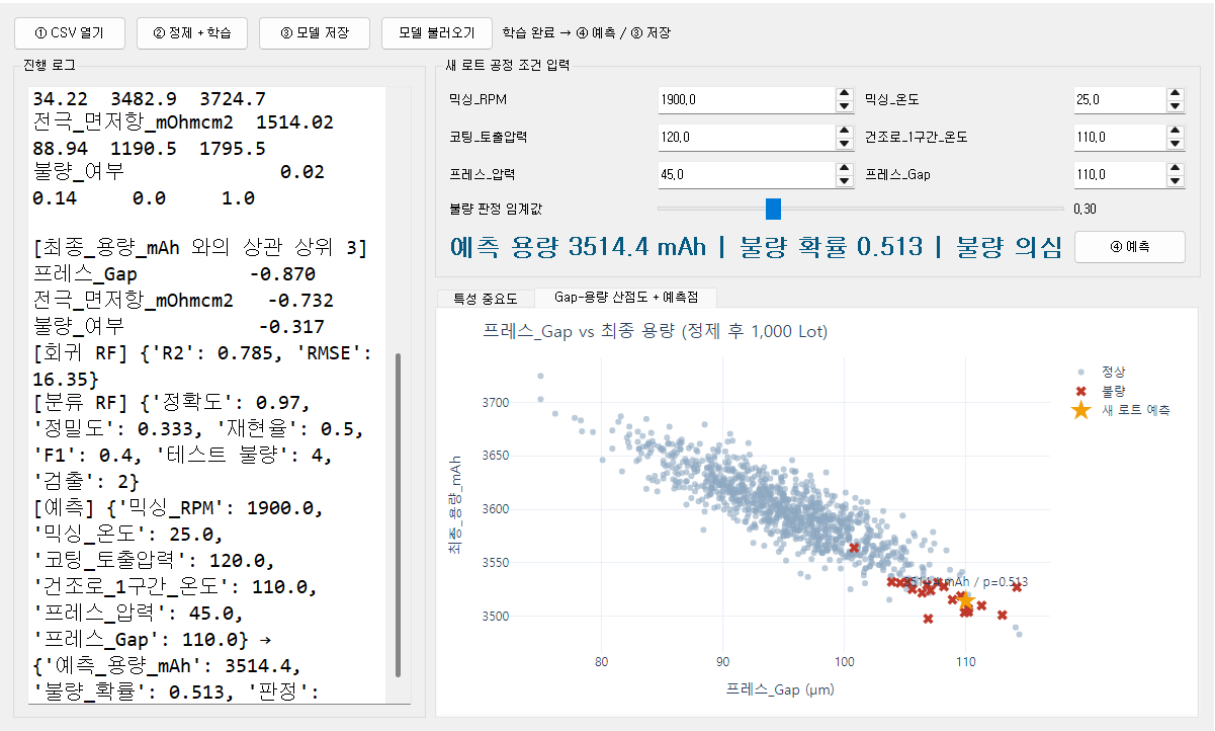
Task

오른쪽 아래에 탭 2개(특성 중요도 막대, 프레스_Gap-최종 용량 산점도 + 예측점)를 추가해라.

제약사항

- Plotly 그림을 QWebEngineView 에 표시
- 불량은 빨간 x, 예측점은 주황 별
- QWebEngine import 는 QApplication 생성보다 먼저

목표 결과물 - 이렇게 나오면 성공



- 다른: 차트가 비어 있으면 → 인터넷 확인, 오프라인이면 "plotly.js 를 HTML 에 포함" 을 제약에 넣는다

4부

조립과 실행, 확장

프롬프트 13~14 · 15분 · main_qt.py
결과물 저장 · 디버깅 요령

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 13. 실행 파일로 조립

파일: main_qt.py

[Gemini 에 입력]

Context

파일 5개를 하나의 프로그램으로 실행한다.

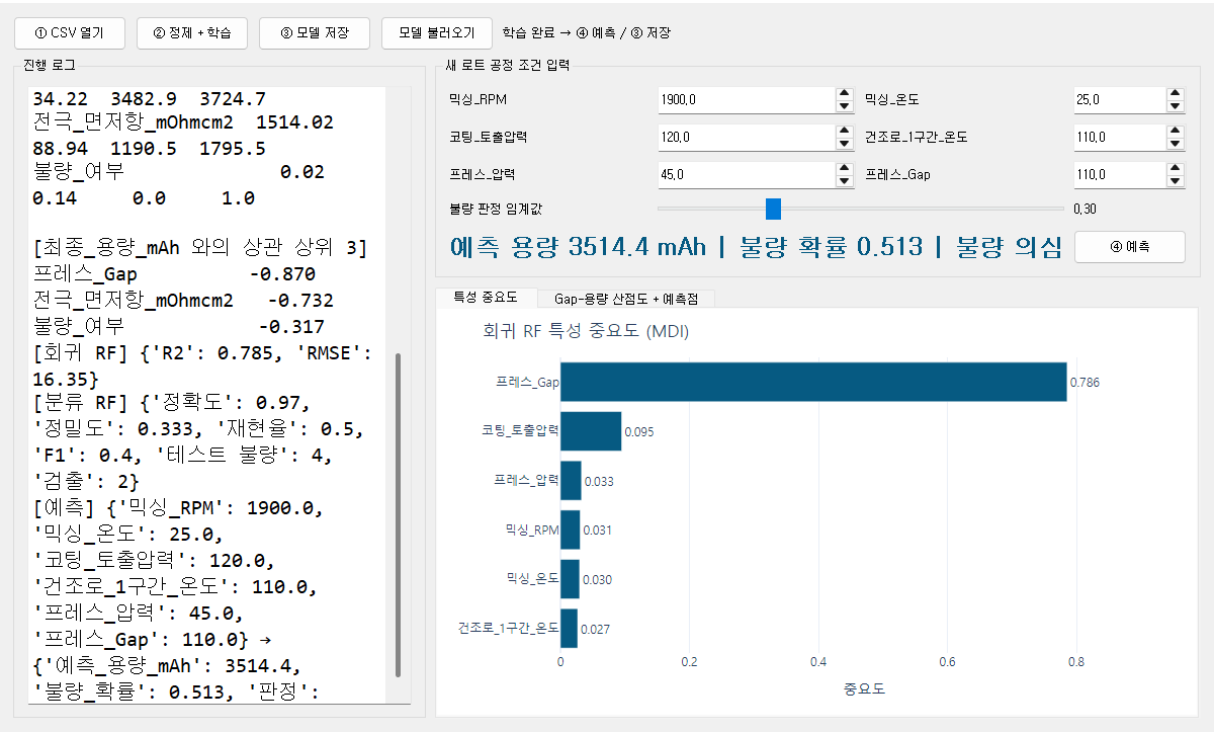
Task

main_qt.py 를 작성해라.

제약사항

- 실행 진입점만(3~5줄)
- gui_qt 를 QApplication 생성 전에 import

목표 결과물 - 이렇게 나오면 성공



- 다른면: ImportError(QtWebEngineWidgets) 가 나면 → import 순서 제약을 그대로 다시 붙인다

체크포인트 ③ - 화면에서 동작 확인

main.py 실행 후 아래 순서로 눌러 본다

순서	동작	화면에서 보여야 하는 것
1	python main_qt.py	1200×720 창, 버튼 4개, 빈 로그·폼·Plotly 탭 2개
2	① CSV 열기 → battery_process_data.csv	로그: 행 1000개 ... 불량 19건 / 상태: 불러오기 완료
3	② 정제 + 학습	로그: 정제 결과·기술통계·회귀·분류 수치 / 차트: 프레스_Gap 막대 1위
4	④ 예측 (기본값)	"예측 용량 3593.1 mAh 불량 확률 0.000 정상" / 산점도 탭으로 전환, 별 마커가 Gap 95 정상 군집 위
5	프레스_Gap 을 110, 믹싱_RPM 을 1900 으로 → ④	용량 약 3510, 불량 확률 상승, 임계값 0.3 이면 "불량 의심"
6	슬라이더를 0.6 으로 → ④	같은 입력이 "정상" 으로 바뀜(임계값의 의미)
7	③ 모델 저장 → 창 닫기 → 재실행 → 모델 불러오기 → ④	학습 없이 같은 예측이 나오고 차트가 그려짐

[표 X-7] 체크포인트 ③ 동작 확인

막혔을 때 - 디버깅 프롬프트와 자주 나는 오류

오류는 전체 메시지를 그대로 붙여넣는다

[디버깅 프롬프트 템플릿]

아래 명령을 실행했더니 오류가 났다.
명령: python main_qt.py
오류 전체:
(여기에 Traceback 전체 붙여넣기)

관련 파일은 gui_qt.py 다. 원인을 한 줄로
설명하고,
고친 gui_qt.py 전체를 다시 출력해라.
다른 파일은 바꾸지 마라.

— 숫자가 다를 때 —
교재는 정제 후 결측 0칸인데 내 결과는 5칸이다.
clean_data 의 어느 단계가 다른지 찾아 고쳐라.

증상	원인	대처
ModuleNotFoundError: No module named 'data_io'	파일이 다른 폴더에 있거나 이름이 다름	5개 파일을 한 폴더에, 파일명 정확히
FileNotFoundError: battery_process_data.csv	작업 디렉터리가 다름	① 버튼으로 파일을 직접 고르거나 절대경로
ImportError: QtWebEngineWidgets must be imported before a QApplication	import 순서	gui_qt(QWebEngineView) 를 QApplication 생성보다 먼저 import
차트 영역이 하얗게 비어 있음	plotly.js CDN 접속 불가(오프라인)	fig_html 에서 include_plotlyjs=True
ModuleNotFoundError: PyQt5.QtWebEngineWidgets	PyQtWebEngine 미설치	pip install PyQtWebEngine (PyQt5 와 같은 5.15.x)
재현율 0.000, 검출 0/4	임계값 0.5 가 너무 높음(불량 2%)	슬라이더를 0.3 이하로(교재 4.13)
ValueError: could not convert string to float	입력 칸에 문자·빈칸	숫자만 입력, 예외 처리 확인
Gemini 가 열 이름을 영문으로 바꿈	맥락 유실	프롬프트 1 을 다시 붙여넣고 '열 이름은 한국어 그대로' 명시

[표 X-8] 자주 나는 오류

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

프롬프트 14. CSV 일괄 예측

파일: gui_qt.py · model.py

[Gemini 에 입력]

Context

하루치 로트 전체를 한 번에 판정하고 싶다.

Task

"CSV 일괄 예측" 버튼을 추가해 정제 후 모든 행을 예측하고 예측결과.csv 로 저장해라.

제약사항

- 원본 열 + 예측_용량_mAh + 불량_확률 + 판정
- 행 단위 반복 대신 한 번에 예측
- 상위 5행을 로그에

목표 결과물 - 이렇게 나오면 성공

예측결과.csv 저장 (1000행 × 13열)

Lot_ID	프레스_Gap	예측_용량_mAh	불량_확률	판정
L25000	93.5	3606.2	0.000	정상
L25001	93.2	3601.9	0.007	정상
L25002	90.8	3627.6	0.000	정상
L25003	97.6	3592.1	0.000	정상
L25004	96.4	3581.8	0.000	정상

■ 다른면: 더 해 볼 것: F1 이 최대인 임계값 자동 추천 · pyinstaller 로 exe 만들기

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

Gemini 사용 시 주의 - 생성 코드는 내 책임

회사 데이터는 올리지 않는다, 숫자는 교재로 검증한다

- 실제 공정 데이터·설비 태그·레시피를 프롬프트에 붙여넣지 않는다 - 이 실습은 공개 실습 CSV 만 쓴다(열 이름 정도만 전달)
- 생성 코드는 반드시 실행하고 교재 수치와 대조한다 - 그럴듯한 코드가 조용히 다른 결과를 내는 경우(정제 순서, stratify 누락, predict vs predict_proba)가 가장 위험
- 라이브러리·버전 차이로 숫자가 소수점에서 다를 수 있다 - 방향(순위, 증감)이 같은지 본다
- Gemini 가 임의로 바꾸기 쉬운 것: 열 이름 영문화, 기본값(`n_estimators=100`), 임계값 0.5 고정, GUI 프레임워크 - 프롬프트에 명시된 값으로 되돌린다
- 한 프롬프트에 기능 하나. "프로그램 전체를 만들어 줘" 는 디버깅이 불가능한 덩어리를 만든다

Key Takeaways

- 3장의 정제 규약과 4장의 Random Forest 가 함수 몇 개로 정리되면, GUI 는 그것을 부르는 버튼일 뿐이다
- 프롬프트는 역할·맥락·제약·출력 형식·검증 기준 - 특히 검증 기준(교재 수치)이 있어야 틀린 코드를 걸러낸다
- 임계값 슬라이더 하나로 4.13 절의 교훈(정확도 97% 인데 불량 0건 검출)을 화면에서 직접 본다
- 특성 중요도 차트는 5.9 절과 같은 순위 - 프레스_Gap 이 용량 산포의 주범
- 다음 단계: 확장 과제 A~C, 또는 6장 이상탐지(Isolation Forest)를 같은 방식으로 탭 하나 추가

번외 실습. Gemini 프롬프트로 조립하는 배터리 품질 예측 GUI (PyQt5 + Plotly 판)

부록 - 강사용 정답 코드와 배포 파일

수강생 배포: prompts.md 만 / 강사: 정답 5개 파일

경로	내용
SecondaryBattery-AI-Example/번외_Gemini_GUI/prompts.md	이 텍스트의 프롬프트 14개 원문(복사용)
SecondaryBattery-AI-Example/번외_Gemini_GUI/check_data.py · check_model.py	체크포인트 ①·② 확인 스크립트(수강생 배포)
SecondaryBattery-AI-Example/번외_Gemini_GUI/data_io.py · cleaning.py · model.py · gui_qt.py · main_qt.py	강사용 정답 코드(이 PC 에서 실행 검증: R ² 0.785, RMSE 16.35, 임계값 0.3 재현율 0.5)
SecondaryBattery-AI-Example/data/battery_process_data.csv	실습 데이터(교재 부록 A 명세)
교재 3.7.1 · 4.4 · 4.12 · 4.13 · 5.9 · 5.13	각 프롬프트의 근거 절 - 수치 대조용

[표 X-4] 배포 파일

- 정답 코드는 수업 중에는 보여 주지 않는다 - 체크포인트에서 막힌 수강생에게 해당 파일만 diff 로 비교
- 실행 검증 환경: Python 3.11 / pandas 2.0 / scikit-learn 1.3. 교재 기준 환경(3.13 / 3.0.5 / 1.9)에서는 소수점이 다를 수 있다